

DAY 9

DICTIONARIES, NESTING AND THE SERIAL AUCTION

Goals:

- However, there is also a type of auction called a [Silent Auction](#), where everyone bids silently, and no one knows the other person's bid until the very end of the auction, when the highest bid is revealed.
- Today's [final program](#)
- Today's main focus is the python dictionaries, we'd also build further on nesting(nest within nests, dictionaries within dictionaries, and mixing nests and dictionaries)

The Python Dictionary: Deep Dive

- the way to conceptualize dictionaries is in the form of a table, every dictionary has two parts:
 - the key; which is equivalent to a word in the dictionary
 - and the associated **value** which is the definition

Creating A Dictionary

- Suppose you have a simple table of programming word definitions. To convert it into a Python dictionary, you use curly braces. The key goes first, followed by a colon, and then the value;

Python Code Sample

 Copy

```
programming_dictionary = {  
    "Bug": "An error in a program that prevents the program from running as expected."  
}
```

- To add more entries, separate each key-value pair with a comma. Continue adding pairs until the dictionary is complete

Formatting Dictionaries for Readability

- When creating a dictionary with multiple elements, formatting for readability is really important

Python Code Sample

 Copy

```
programming_dictionary = {  
    "Bug": "An error in a program that prevents the program from running as expected.",  
    "Function": "A piece of code that you can easily call over and over again.",  
    "Loop": "The action of doing something over and over again."  
}
```

- create the start and ending curly braces on different lines, just like in C.
- Indent each entry on different lines
- always add a comma “,” at the end of your key-value pair, even if there’s no new entry, this makes it easier to add more items later

Retrieving Items from a Dictionary

- To retrieve an item from a dictionary, use square brackets and provide the key.

```
print(programming_dictionary["Bug"])
```

- it is important to spell the key properly, and be key sensitive, a common error is a typo in the key which would lead to a key error

Data Types for Keys

- When defining a dictionary, ensure that keys are the correct data type. If you omit the quotes for string keys, Python will interpret them as variable names, resulting in an error. When retrieving data, provide the key in its correct data type.

Adding and Wiping Dictionary Entries

- To add a new entry to a dictionary use square brackets with the new key and assign a value

```
programming_dictionary["Loop"] = "The action of doing something over and over again."
```

- You can also create an empty dictionary using curly braces with nothing inside. To wipe an existing dictionary, assign it to an empty dictionary.

```
programming_dictionary = {}
```

this is useful for clearing user progress or resetting scores in a game

Editing Dictionary Entries

- To edit an item in a dictionary, use the same syntax as adding a new item. If the key exists, its value will be updated; if not, a new entry is created.

Looping Through A Dictionary

- To loop through a dictionary, use a for loop. By default, this iterates over the keys.

```
for key in programming_dictionary:  
    print(key)
```

- running the code above will print the key, to print the value:

```
21     for key in programming_dictionary:  
22         print(key)  
23         print(programming_dictionary[key])
```

Coding Exercise: Grading Program

- unlike in lists you can use the keys to create another set of keys in another dictionary, while using the previous content of the previous dictionary to create the content of the new dictionary,
- before adding content to a new dictionary through a loop, try to initialize the new dictionary(or create your new dictionary as empty)
- you can store a value inside a variable for readability and ease of use, although that doesn't equal the key to your variable. In cases of a loop it spares you the stress of calling the key to compare conditions each time, instead you just call the variable for each comparison
- **My code lacks comments, i should really work on that**

Nesting Lists and Dictionaries

- nesting lists and dictionaries are just like stacking folders inside other folders
- you can use lists and dictionaries as values inside dictionaries, **you can use multiple data types as values for different keys in a dictionary, all keys musn't necessarily have same data types**
 - **keys must be immutable,i.e. that is like str, integers, floats, booleans where you can't change it's content, but you can't use thing like lists and other dictionaries**
- to call or use the a specific item of a list that is a value of a specific dictionary: `print(travel_log["France"][0])` , **you call the value with the first square bracket, and then the item of that value with the next square bracket**
- you can also nest lists within lists, to call an item from within a nested list you have to take the same approach as with dictionaries; `print(nested_list[2][1])` , **you call the item list using the 1st square bracket, and using the 2nd square bracket to call the item within the item list**
- to access dictionaries within dictionaries, call the dictionaries in order using square brackets

```
19 travel_log = {
20     "France": {
21         "cities_visited": ["Paris", "Lille", "Dijon"],
22         "total_visits": 12
23     },
24     "Germany": {
25         "cities_visited": ["Berlin", "Hamburg", "Stuttgart"],
26         "total_visits": 5
27     },
28 }
29 print(travel_log["Germany"]["cities_visited"][2])
```

The Secret Auction Program Instructions and Flowchart

- Information on the [secret auction](#)
- [demo](#)
- the program has to clear out each bid before asking for the next, unless there's no one else before giving the winner of the next bid
 - for now she doesn't expect us to know how to clear out the previous inputs, so instead after a bid we multiply the newline character by say 100 to make quite the space between the previous bid and the next
- we are to use dictionaries: where the bidder will be the key, and their bid will be the value
- try creating your own [flowchart](#) before taking a look at [hers](#)
- creating a new dictionary, is not the same as adding new content to an existing dictionary, if you just do;
 - `bid_dict = [`
 `name: bid`
 `]` , this will create the same list anew over and over again
 - `bid_dict[name] = bid` , this will redefine a certain key if it exists, or create a new one
- to create a todo comment, just write `#TODO` in capital letters and write what's to do
- generate your todos' according to your flowchart you've created
- you can actually use the f strings to replace the `format()` function, you can directly just put `:.2f` ; e.g. `print(f"The winner is {key} with a bid of: ${bid_dict[key]:.2f}")`
- remember the output of the `format()` function is a string
- if anything is initialized or set to a certain value within a while loop, anytime the loop restarts all the previous values will be lost.
- When working on multiple functionalities within a single project, it's adviceable to define a function per functionality that way you can use it as many times as you might need, and helps with readability
- the `max()` function can also be used on a dictionary
- `max(data)` by default returns the key that is "maximum" in alphabetical order.
- `max(data, key=data.get)` tells Python to return the key whose **value** is the highest, and you can simply use the key to get the value; `data[key]` and then store it in a variable
- it doesn't make sense to use the `max()` function with a loop as it's using one temporary item as a variable at a time, so each temporary item or value is a temporary max